Open OAuth2TokenStore.java in your editor and update the constructor to configure secure TLS settings.

Listing 7.8 Securing the HTTPS connection

```
import javax.net.ssl.*;
import java.security.*;
import java.net.http.*;

var sslParams = new SSLParameters();
sslParams.setProtocols(    #A
        new String[] { "TLSv1.3", "TLSv1.2" });    #A
sslParams.setCipherSuites(new String[] {
        "TLS_AES_128_GCM_SHA256",    #B
        "TLS_AES_256_GCM_SHA384",    #B
        "TLS_CHACHA20_POLY1305_SHA256",    #B

        "TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256",    #C
        "TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256",    #C
        "TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384",    #C
        "TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384",    #C
        "TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256",
#C
        "TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256"    #C
});
sslParams.setUseCipherSuitesOrder(true);
sslParams.setEndpointIdentificationAlgorithm("HTTPS");

try {
    var trustedCerts = KeyStore.getInstance("PKCS12");    #D
    trustedCerts.load(    #D
            new FileInputStream("as.example.com.ca.p12"),
#D
            "changeit".toCharArray());    #D
    var tmf = TrustManagerFactory.getInstance("PKIX");    #D
    tmf.init(trustedCerts);    #D
    var sslContext = SSLContext.getInstance("TLS");    #D
```